

ELI — Dag

ELI v2.5.9

October 2026

Contents

ELI DAG Engine — project-wide dependency scheduling	1
The engine (eli/core/dag.py)	1
Consumer 1 — the agent bus runs on the DAG (eli/cognition/agent_bus.py)	1
Consumer 2 — the coding engine’s subtask DAG (eli/coding/plan_graph.py)	2
Consumer 3 — MULTI_COMMAND’s dependency graph (eli/execution/command_dependency_graph.py, 2026-09-29)	2
Scope & honesty	2
Control summary	3

ELI DAG Engine — project-wide dependency scheduling

A single, generic directed-acyclic-graph engine (eli/core/dag.py) is now the shared scheduling primitive across ELI: the **agent bus** runs its agents on it, the **coding engine** runs its subtasks on it, and **MULTI_COMMAND** orders chained commands on it. One DAG algorithm, three consumers — no per-subsystem reinvention.

The engine (eli/core/dag.py)

Pure, deterministic, no I/O — fully unit-tested.

- DAG / DAGNode — nodes with depends_on edges ($B \text{ depends_on } A \Rightarrow A \rightarrow B$).
- topological_order() — Kahn’s algorithm, deterministic (sorted ties); raises DAGCycleError on a cycle.
- topological_layers() — groups nodes into **parallel layers**: layer k holds every node whose dependencies are all in earlier layers. This is what lets independent work run concurrently while dependent work waits.
- ancestors() / descendants() — transitive closure.
- critical_path_length() — longest dependency chain = number of sequential layers (a parallelism metric).
- build_dag({node: [deps]}) — convenience builder that **drops deps outside the given node set**, so a global dependency map can be applied to any subset safely.

Consumer 1 — the agent bus runs on the DAG (eli/cognition/agent_bus.py)

Previously the bus was a single-round star fan-out (agents couldn’t use each other’s output). Now:

- _AGENT_DEPENDENCIES declares edges among agents. Shipped edges: knowledge_graph $\leftarrow$ memory, and critic $\leftarrow$ {memory, system, knowledge_graph, file_code} — CriticAgent is the bus’s second-tier verifier: it reasons over what its upstream retrievers found (via intent["_upstream"]) rather than the raw query, measuring pairwise term-overlap agreement between them and downgrading

confidence on contradiction. It self-gates to nothing when fewer than 2 upstream sources are available, so it never fires on trivial turns.

- `_agent_execution_layers(active_names)` builds a DAG from that map $\cap$ the selected agents and returns topological layers.
- `AgentBus._run_agents_layered` runs **layer by layer**: each layer's agents run in parallel (same per-agent hard timeout + failure handling as before), then every completed layer's results are passed to downstream agents via `intent["_upstream"]`. The `knowledge_graph` agent uses this to seed its lookup with what memory surfaced this turn.
- **Non-regressive**: when no edges apply to the selected set, the DAG collapses to a single layer = the legacy flat fan-out. Gated by `ELI_AGENT_DAG` (default on); any error falls back to flat dispatch (`_run_agents_flat`).

selected agents $\rightarrow$ `_agent_execution_layers` $\rightarrow$ `[[memory, reflection, ...], [knowledge_graph], ...]`
| run parallel | receives `_upstream={memory:...}`

Add an edge by editing `_AGENT_DEPENDENCIES`; cycles are rejected at build time.

Consumer 2 — the coding engine's subtask DAG (`eli/coding/plan_graph.py`)

For decomposable coding tasks:

- `decompose_dag(task, generate)` asks the planner for a **graph** of subtasks (`{nodes:[{id, task, depends_on}]}`), builds a DAG, and validates it (cycle/ dangling-edge safe). A cohesive task yields a single node.
- `solve_dag(...)` walks the topological order, solving each node via the normal single-shot tree search and **feeding each node the code of its dependencies**, then `compose()` concatenates the node solutions (imports hoisted/deduped) into one module. `CodeAgent.solve` verifies the composed module against tests synthesized for the *original* task.
- Single-node tasks return `None` so the agent uses the plain single-shot path (no DAG overhead). Gated by `ELI_CODING_DAG` (default on).

Consumer 3 — MULTI_COMMAND's dependency graph (`eli/execution/command_dependency_g` 2026-09-29)

For a chained command request ("close steam and set an alarm" / "open the file and then edit it"):

- `infer_dependencies(commands)` mirrors `plan_graph.decompose_dag`'s shape: one optional LLM call asking only for dependency EDGES among the fixed, already-split command list (not a re-decomposition), validated through `build_dag(...).topological_layers()`. Falls back to fully independent (empty dict — today's behavior) on no model, bad JSON, an out-of-range index, or a cycle.
- The executor runs `MULTI_COMMAND` in topological layers. A step whose dependency didn't succeed is skipped and reported as such (not silently run); independent branches still run regardless of an unrelated failure. Gated by `ELI_MULTI_COMMAND_DAG` (default on).
- Deliberately narrower than consumers 1-2: it only asks for edges among an already-fixed list, never regenerates or reorders the commands themselves — the router's `command_splitter.py` owns splitting, this only owns ordering.

Scope & honesty

- **DAG-driven now**: agent-bus execution (topological layers + upstream), coding subtask decomposition, and `MULTI_COMMAND` dependency-aware execution all run on `eli/core/dag`.
- **Still linear (not yet DAG)**: the 12-stage orchestrator pipeline is a fixed chain; `execution_planner.ExecutionPlan.steps` is an ordered list. Converting those to DAGs is a possible next step but wasn't required for "agents on a DAG".
- **Composition is v1**: the coding DAG composes node code by ordered concatenation with import-deduping; integration quality scales with the model.

- Everything here is covered by tests/test_dag.py (engine, layered dispatch via fake agents, coding subtask DAG) plus tests/test_multi_command_dependencies.py (consumer 3) — all deterministic, no real model calls in the test suite.

Control summary

Knob	Default	Effect
ELI_AGENT_DAG	on	agent-bus topological layered dispatch (off ⇒ flat fan-out)
ELI_CODING_DAG	on	coding subtask-DAG decomposition (off ⇒ single-shot only)
ELI_MULTI_COMMAND_DAG	on	MULTI_COMMAND dependency inference (off ⇒ fully independent)
_AGENT_DEPENDENCIES	{knowledge_graph: {memory}, critic: {memory, system, knowledge_graph, file_code}}	declare agent edges